

FIA

FIA	1
Partie I : Le jeu, l'ultimate Tic Tac Toe :	2
1. Introduction et Architecture Globale.....	2
2. Le Choix du Binaire (L'approche "Bitboard")	2
3. Fonctionnement Détaillé des Composants	3
4. Conclusion	4
Partie II : l'évaluation	5
generator.cpp	5
train.py	5
neural_net.h et neural_net.cpp	5
Résultats	6
Partie III : le Minimax	6
1. Introduction et Objectif	6
2. Conditions d'Arrêt et Évaluation (Les Feuilles de l'Arbre)	6
3. La Table de Transposition (TT)	7
4. Alpha-Bêta et Récursivité.....	7
5. L'Ordonnancement des Coups (order_moves).....	8
6. Mise en Cache (Stockage TT).....	8
Conclusion	8

Partie I : Le jeu, l'ultimate Tic Tac Toe :

1. Introduction et Architecture Globale

Ce document détaille l'architecture d'un moteur de jeu pour l'Ultimate Tic-Tac-Toe en C++.

Le code est structuré autour de deux composants principaux :

- **L'espace de noms BoardHandling** : Contient les fonctions de bas niveau permettant de manipuler les grilles individuelles (3x3).
- **La classe Game** : Gère l'état global de la partie (les 9 sous-grilles, la grande grille, le trait, l'historique) et l'interfaçage avec un réseau de neurones (NeuralNet) pour l'évaluation des positions.

2. Le Choix du Binaire (L'approche "Bitboard")

La caractéristique la plus marquante de ce code est l'utilisation des opérations binaires (bit à bit) pour représenter et manipuler le jeu. Au lieu d'utiliser des tableaux classiques (ex: `int board[9][9]`), l'état de chaque grille 3x3 est encodé dans un seul entier (le type `board`).

Puisqu'une grille fait 9 cases, seuls 9 bits sont nécessaires. La grille complète (`FULL_MASK`) est représentée par `0x1FF` (qui correspond à 11111111 en binaire).

Pourquoi ce choix ?

- **Vitesse d'exécution fulgurante** : Les processeurs sont conçus pour traiter les opérations binaires (AND, OR, XOR, NOT) en un seul cycle d'horloge. Vérifier si un coup est valide, placer un pion ou vérifier une victoire prend une fraction de nanoseconde.
- **Vérification de victoire instantanée** : Plutôt que de parcourir les lignes et les colonnes avec des boucles for, la fonction `HasWon` utilise 8 "masques" précalculés représentant les lignes gagnantes (ex: `0x007` pour la ligne du haut). Une simple opération (`player & masque == masque`) suffit pour savoir si un joueur a gagné.
- **Génération des coups ultra-rapide** : Pour trouver les cases vides, le code fait un simple `~(playerBoard | OpponentBoard) & FULL_MASK`. Ensuite, l'instruction intrinsèque `_BitScanForward` est utilisée. C'est une commande matérielle du processeur qui trouve instantanément la position du premier bit à 1, évitant ainsi de devoir itérer sur chaque case.

- **Empreinte mémoire minimale** : Tout l'état du plateau (joueur X et joueur O) tient dans quelques octets, ce qui permet au processeur de garder les états du jeu dans ses caches de niveau 1 (L1 cache), accélérant drastiquement les calculs.

3. Fonctionnement Détaillé des Composants

Gestion de l'État Global (Game) :

La classe maintient l'état via plusieurs variables :

- `subX[9]` et `subO[9]` : Tableaux contenant l'état des 9 sous-grilles pour chaque joueur.
- `bigX` et `bigO` : Entiers représentant la grille principale (qui a gagné quelle sous-grille).
- `activeBoard` : Indique sur quelle sous-grille le joueur actuel doit jouer (-1 si le choix est libre).
- `currentPlayer` : Alterne entre les joueurs après chaque coup.

Mécanique de Jeu (`apply_move` et `undo`) :

Le code est conçu pour explorer l'arbre des possibles :

1. **Jouer un coup (`apply_move`)** : La fonction enregistre d'abord l'état actuel dans `moveHistory`. Elle met ensuite à jour la sous-grille concernée. Si ce coup fait gagner la sous-grille, le "Big Board" (`bigX` ou `bigO`) est mis à jour. Enfin, elle détermine la prochaine `activeBoard` (qui devient -1 si la grille cible est déjà pleine/finie).
2. **Annuler un coup (`undo`)** : Indispensable pour les algorithmes d'IA (comme l'Alpha-Beta). Au lieu de devoir recréer une copie du plateau à chaque mouvement, on restaure simplement l'état précédent stocké dans la liste `moveHistory`, ce qui économise énormément de ressources mémoire.

Le Hachage de Zobrist (`get_hash`) :

La méthode `get_hash` implémente un système de clés de Zobrist.

- **Le principe** : Chaque combinaison de pion sur chaque case, ainsi que le trait et le plateau actif, se voient attribuer un nombre aléatoire sur 64 bits à l'initialisation

(`init_zobrist`). L'identifiant du plateau est obtenu en appliquant l'opération XOR sur ces nombres.

- **L'utilité :** Cela crée une "empreinte digitale" unique (un hash) pour n'importe quel état de jeu. C'est crucial pour créer des tables de transposition (Transposition Tables), permettant à une IA de se souvenir si elle a déjà évalué cette position exacte dans une autre branche de son arbre de recherche.

L'Intégration du Réseau de Neurones

La méthode `get_state()` transforme le plateau compact (binaire) en un format compréhensible par un modèle d'apprentissage automatique (Machine Learning). Elle génère un tenseur "aplati" de 406 valeurs (des floats valant 1.0 ou 0.0), organisé en "plans" (planes) :

1. Les pièces du joueur actuel (81 cases).
2. Les pièces de l'adversaire (81 cases).
3. Les sous-grilles remportées par le joueur (81 cases).
4. Les sous-grilles remportées par l'adversaire (81 cases).
5. Les cases constituant des cibles valides pour le prochain coup (81 cases).
6. Un scalaire indiquant à qui c'est le tour (1 case).

Cette représentation est classique dans le domaine des IA modernes (inspirée d'AlphaZero), car elle permet au réseau de neurones de "voir" les relations spatiales et les règles de placement.

4. Conclusion

Ce code n'est pas une simple récréation des règles de l'Ultimate Tic-Tac-Toe, c'est **un environnement de simulation hautement optimisé**.

Le choix d'utiliser des variables binaires (`bitboards`) démontre une volonté de maximiser l'efficacité. Toute l'architecture (système d'historique undo, hachage de Zobrist, tenseurs d'état) est faite pour la préparation d'un environnement destiné à l'entraînement ou à l'exécution d'une Intelligence Artificielle.

Partie II : l'évaluation

Concrètement, le minimax alpha-bêta a besoin de savoir si une position est bonne ou mauvaise sans explorer tout l'arbre. C'est ma fonction qui lui donne cette information — elle prend l'état du plateau et retourne un score entre par rapport à divers signal du plateau notamment l'avantage sur chaque plateau.

Pour faire ça j'ai entraîné un réseau de neurones sur toutes les parties possibles du petit Tic-Tac-Toe classique, puis je l'ai intégré en C++ pour que le minimax puisse l'appeler directement dans son code.

generator.cpp

Ce fichier génère toutes les parties possibles du Tic-Tac-Toe par force brute. J'utilise une fonction récursive qui essaie tous les coups possibles à partir d'une position vide. À chaque fois qu'une partie se termine (victoire X, victoire O ou match nul), j'enregistre l'état du plateau dans un fichier CSV avec le résultat final. J'ai réutilisé les fonctions déjà existantes dans board.h comme MakeMove, HasWon et hasTied. Le CSV généré contient environ 255 000 lignes.

train.py

Ce fichier charge le CSV et entraîne un réseau de neurones avec PyTorch. Le réseau prend en entrée les 9 cases du plateau (1 pour X, -1 pour O, 0 pour vide) et retourne un score entre -1 et +1. J'ai utilisé une architecture simple avec 3 couches et des fonctions ReLU entre les couches. Après 100 passages sur toutes les données, le réseau a appris à reconnaître les positions gagnantes et perdantes. Les poids appris sont exportés dans un fichier JSON pour pouvoir être utilisés en C++.

neural_net.h et neural_net.cpp

Ces fichiers reconstituent le réseau de neurones en C++ sans avoir besoin de PyTorch. Au démarrage du programme, loadWeights charge le fichier JSON et met les poids en mémoire. Ensuite, evaluate(p1, p2) prend les deux plateaux en paramètre, les convertit en

vecteur de 9 valeurs, fait passer ces valeurs à travers les 3 couches du réseau et retourne le score final entre -1 et +1. C'est cette fonction que Pierre appelle dans son minimax.

Résultats

Après entraînement, j'ai testé mon évaluateur sur quelques positions :

- Position vide $\rightarrow$ score ≈ 0 (personne n'a d'avantage)
- X a gagné $\rightarrow$ score $\approx +0.96$ (très proche de +1)
- O a gagné $\rightarrow$ score ≈ -0.98 (très proche de -1)

Ces résultats montrent que le réseau a bien appris à distinguer les positions gagnantes et perdantes.

Partie III : le Minimax

1. Introduction et Objectif

Ce document analyse la fonction `minimax`, qui constitue le « cerveau » de l'Intelligence Artificielle pour l'Ultimate Tic-Tac-Toe. Alors que le code précédent (en C++) fournissait les règles de l'univers et la mécanique de mouvement, cette fonction explore l'arbre des possibles pour prendre la meilleure décision.

2. Conditions d'Arrêt et Évaluation (Les Feuilles de l'Arbre)

L'algorithme commence par vérifier s'il doit stopper son exploration dans deux cas précis :

- **État Terminal (`game.is_done()`)** : Si la partie est finie (victoire, défaite ou égalité), la fonction retourne un score absolu (`WIN_SCORE`, `LOSS_SCORE` ou `0`). C'est la vérité absolue du jeu.
- **Profondeur Maximale Atteinte (`depth == 0`)** : Si l'IA ne peut plus "réfléchir" plus loin, elle fait appel à son intuition via `game.eval()`.
 - *Pourquoi ce multiplicateur de 900 000 ?* Cela permet de s'assurer que les scores d'évaluation heuristique (basés sur le réseau de neurones) restent toujours strictement inférieurs aux scores de victoire absolue (qui valent probablement des millions), garantissant que l'IA préférera toujours une victoire immédiate à une position "stratégiquement avantageuse".

3. La Table de Transposition (TT)

des jeux complexes, on peut atteindre exactement la même configuration de plateau par des séquences de coups différentes (des transpositions).

- **La Clé Unique :** Vous utilisez `game.get_hash()` (le hash de Zobrist préparé dans le code C++) combiné au trait (`maximizing`) pour créer une `state_key` unique.
- **La Récupération (Lookup) :** Avant de calculer quoi que ce soit, le code vérifie si cette position a déjà été explorée (`_tt.get`). Si oui, et si elle a été explorée à une profondeur égale ou supérieure (`cached_depth >= depth`), on réutilise l'information :
 - `_EXACT` : On connaît la valeur exacte, on la retourne directement.
 - `_LOWER` / `_UPPER` : Si la recherche précédente avait été coupée prématurément, on utilise ces bornes pour resserrer la fenêtre `alpha / beta`. Si la fenêtre se ferme (`alpha >= beta`), on gagne un temps de calcul précieux en retournant la valeur en cache.

4. Alpha-Bêta et Récursivité

Si la position est inconnue, l'algorithme doit l'explorer. Il se divise en deux comportements selon si l'IA cherche à maximiser son score (`maximizing == True`) ou si elle simule le tour de l'adversaire qui cherche à le minimiser (`False`).

- **Synergie avec l'architecture (`apply_move` / `undo`) :** On retrouve ici l'utilité du design C++. Au lieu de cloner le plateau complet à chaque nœud (ce qui saturerait la mémoire), le code joue un coup `apply_move`, lance la recherche récursive `minimax(...)`, puis annule le coup `undo()`. C'est l'approche la plus rapide possible.
- **L'Élagage Alpha-Bêta (Pruning) :** * `alpha` représente le score minimum assuré pour le joueur "Max".
 - `beta` représente le score maximum assuré pour le joueur "Min".
 - Dès que le code détecte que `value > alpha` (ou `value < beta` pour l'adversaire), il met à jour la borne. Si `alpha >= beta`, la boucle s'arrête net (`break`). L'algorithme a prouvé mathématiquement que l'adversaire ne nous laissera jamais atteindre cette branche, il est donc inutile d'en explorer le reste.

5. L'Ordonnement des Coups (order_moves)

Juste avant la boucle principale, l'appel à `order_moves(moves)` est crucial.

- *Pourquoi l'avoir fait ?* L'élagage Alpha-Bêta est d'autant plus efficace qu'il trouve les "bons" coups tôt. Si l'algorithme explore le meilleur coup en premier, il va maximiser `alpha` (ou minimiser `beta`) immédiatement, ce qui provoquera des break massifs sur tous les autres coups explorés ensuite. Sans ce tri, le gain de performance de l'Alpha-Bêta serait purement aléatoire.

6. Mise en Cache (Stockage TT)

À la fin de l'évaluation d'un nœud, le résultat est sauvegardé pour le futur. Le code détermine intelligemment le type de drapeau (`flag`) :

- Si le score est resté inférieur ou égal à l'`alpha` de départ, on n'a pas trouvé de bon coup : c'est une borne supérieure (`_UPPER`).
- Si le score a causé une coupure bêta (`value >= beta`), le score réel pourrait être encore plus grand : c'est une borne inférieure (`_LOWER`).
- Sinon, on a parcouru tous les coups sans coupure : c'est la valeur exacte (`_EXACT`).

Conclusion

Ce code est une implémentation robuste d'un Minimax "Tournament-Ready". Il tire parfaitement parti des fondations posées par votre moteur de jeu bas niveau (Bitboards et Hachage de Zobrist). L'utilisation conjointe des tables de transposition, du tri des coups et de l'élagage Alpha-Bêta garantit que votre IA explorera l'arbre de manière exponentiellement plus rapide qu'une approche classique, lui permettant de voir plus loin et de jouer beaucoup plus fort.